**Solution 1.** The user should provide $S \subseteq V(D)$, such that $\delta^{\text{out}}(S) = B$ and $\delta^{\text{in}}(S) = \emptyset$, where $B$ is the directed cut. This object must be easy to have when the user provides a directed cut.

**Algorithm 1** Verify-Directed-Cut($D$, $B$, $S$)

```
for each a ∈ A(D) do
    v, u ← ends[a]
    if v ∈ S and u ∉ S then    ▷ This can be done in constant time using an auxiliary array
        if a ∉ B then                                  ▷ Same thing from above comment
            return False
        end if
    else if v ∉ S and u ∈ S then
        return False
    else if v ∈ S and u ∈ S then
        if a ∈ B then
            return False
        end if
    else
        if a ∈ B then
            return False
        end if
    end if
end for
return True
```

**Solution 2.** I will prove that $H_1^{\min} = H_2^{\min}$ by showing that $H_1^{\min} \subseteq H_2^{\min}$ and that $H_2^{\min} \subseteq H_1^{\min}$. I will show that $x \subseteq y$ by proving that an arbitrary element $x_0 \in x$ lies in $y$.

Given $h_1 \in H_1^{\min}$, $h_1$ is a minimal $rs$-cut. By definition, $h_1$ is a set of all the outwards arcs from a set of vertices that includes $r$ and exclude $s$, and therefore $h_1 \subseteq H_2$, because it is impossible to have an $rs$-path if there are no arcs from this set and the other vertices of $V(D)$. $h_1$ is minimal in $H_2$ if every element in $h_1$ is an arc that can be in a different $rs$-path from the other arcs, and it is not an arc that only is in walks that just adds arcs to an $rs$-path with another arc in $h_1$, otherwise it could be removed, and this new set would be a proper subset of $h_1$. First, $h_1$ does not have more than one arc sharing only same $rs$-paths because it would not be minimal in $H_1^{\min}$ given that you could just remove all the arcs besides the one that is further from $r$ and the set would still be an $rs$-cut (a possible shore is to maintain the previous shore and add all the vertices along the paths till the farthest one). Second, there are no arcs in $h_1$ that are part of an extension (loops) to an $rs - path$ with another arc in $h_1$ because you could remove it by having a new shore that adds all the vertices of the loops to a shore associated with the previous $rs$-cut. Finally, it is not possible to have an arc in $h_1$ that leads to path that does not reach $s$ because this is not a minimal $rs$-cut given that you could add all the vertices of those dead end paths to the previous shore to have a new shore associated with an $rs$-cut that is a subset of the previous $rs$-cut. Given all the arguments, $h_1 \subseteq H_2$, and it is minimal in $H_2$, thus $H_1^{\min} \subseteq H_2^{\min}$.

Given $h_2 \in H_2^{\min}$, $h_2$ is a minimal subset $B \subseteq A$ such that $D - B$ has no rs-path. This is an $rs$-cut because you can have a shore that is a set of all the vertices connected from $r$ to the vertices that the outward edges are on B. If those vertices have arcs that are not in $B$, it means that is an arc that does not lead to a new path to $s$ or is part of a loop that goes back to a vertex in the path which the arc that is in $B$ is part of. In both cases, because of the aspects I explained in the last paragraph, you could have a new shore, including all the vertices of the loop and the paths that do not reach $s$, which is associated with a new $rs$-cut without those arcs that are not in $B$. Thus, $h_2 \subseteq H_1$. Also, $h_2$ is minimal in $H_1$ because if you remove an arc $a_2$ from $h_2$, you can not have a shore associated with the $rs$-cut $h_2 - a_2$. This holds because this arc is a unique arc from an $rs$-path in $h_2$. If you add any neighbor of the tail of $a_2$ in a $h_2$ shore to try to remove $a_2$ from $h_2$, you will add at least one arc in $h_2$ because the end of the $rs$-path is $s$ and $s$ can not be in a shore. If you try to remove vertices from a shore associated with $h_2$ you would add at least one arc to $h_2$ because the beginning of the $rs$-path is $r$ and $r$ is in every possible shore. In both cases you can not remove an arc from $h_2$ without adding another arc. Given that, $h_2 \subseteq H_1$, and it is minimal in $H_1$, thus $H_2^{\min} \subseteq H_1^{\min}$.

By the arguments shown, $H_1^{\min} = H_2^{\min}$.

**Solution 3.** The following algorithm solves the problem.

**Algorithm 2** MinimizeWalk($D$, $r$, $s$)

$V_{\text{aux}} \leftarrow \emptyset$ ▷ set of vertices of an auxiliary digraph
$A_{\text{aux}} \leftarrow \emptyset$ ▷ set of arcs of an auxiliary digraph
**for each** $v \in V(D)$ **do**
  $V_{\text{aux}} \leftarrow V_{\text{aux}} \cup (v,0) \cup (v,1) \cup (v,2)$ ▷ new vertices on the auxiliary digraph
**end for**
**for each** $a \in V(A)$ **do**
  $v, u \leftarrow ends[a]$
  $A_{\text{aux}} \leftarrow A_{\text{aux}} \cup ((v,0),(u,1)) \cup ((v,1),(u,2)) \cup ((v,2),(u,0))$
**end for**
$\pi, d \leftarrow$ Breadth-First-Search$((V_{\text{aux}}, A_{\text{aux}}), v)$
**if** $d[s_1] < \infty$ **then**
  $p_{\text{aux}} \leftarrow$ Path-From-Branching$((\pi, d), s)$
  $p \leftarrow$ Relabeling$(p_{\text{aux}})$ ▷ This function will be explained below
  **return** $p$
**end if**
**return** NIL

Using the TA's hint, I will use an auxiliary digraph $D_{\text{aux}} := (V_{\text{aux}}, A_{\text{aux}})$ to solve the problem. $D_{\text{aux}}$ is built from $D$ and based on two rules:

- For each $v \in V(D)$, $V_{\text{aux}}$ has three vertices $v_0, v_1, v_2$, so the size of $V_{\text{aux}}$ is three times the size of $V(D)$.
- For each arc $vu \in A(D)$, $A_{\text{aux}}$ has three arcs $v_0u_1, v_1u_2, v_2u_0$. Also, $A_{\text{aux}}$ is three times the size of $A(D)$.

The algorithm builds the auxiliary digraph, runs a Breadth-First-Search on the auxiliary digraph, and then translate the answer to the desired result.

I will prove the correctness of the algorithm by proving that I can ensure that the $rs$-path the algorithm finds in the auxiliary graph is $rs$-walk that has length $l \equiv 1 \pmod 3$. Also, I have to prove that, if there is an $rs$-walk in $D$ that has length $l \equiv 1 \pmod 3$, we will find it using the provided algorithm. The last statement I will divide in two parts, prove that every walk in $D$ has a correspondent walk in the auxiliary digraph, and that we will find the correspondent path for a $rs$-walk in $D$ that has length $l \equiv 1 \pmod 3$ if it exists.

After building this auxiliary digraph, we run a Breadth-First-Search and verify if there is a path from $r_0$ to $s_1$. The idea to do that is that every path on the auxiliary digraph that ends on a vertex of the form $(v,1)$ is a path where the length $l \equiv 1 \pmod 3$. This is evident because, using the auxiliary digraph, every node that you transverse you go from a vertex of the form $(v,i), i \in \{0,1,2\}$ to a vertex of the form $(u,(i+1)\%3)$, where $\%$ is the operator that gives you the remainder of the division. As for each arc that you traverse the length of the path increases 1, if you start at a vertex of the form $(v,0)$, considering a walk $w$, for each vertex $(v,i), i \in \{0,1,2\}$ of the walk, holds that the length of $w$ is equivalent to $i$ mod 3. By that, if we end on a vertex of the form $(v,1)$, the length $l$ of the path on the auxiliary digraph is equivalent to 1 mod 3.

Also, if there is an $rs$-walk, where the length $l \equiv 1 \pmod 3$, this algorithm will find it. First, every walk in $D$ has a correspondent walk in the auxiliary digraph because if you can go from a vertex $v$ to a vertex $u$ on $D$, you can go from a vertex of the form $(v,i), i \in \{0,1,2\}$ to a vertex of the form $(u,(i+1)\%3)$ in the auxiliary digraph. If you ignore the second element you will have a walk that can be easily converted to the walk in $D$. If a walk in $D$ goes through an arc more than one time in different manners, meaning that, when you go through this arc, the length of the current walk is different from the previous times, this walk is transformed in a path on the auxiliary digraph because the ends of the correspondents arcs of this arc in the auxiliary digraph will be different in the two mentioned steps of the walk. Given that the walk in $D$ that goes

through the same arc more than one time is not relevant for our goal because the walk without the loop and this walk will have the same result in respect to the length mod 3. By that, all the walks we are interested are transformed into paths in the auxiliary graph. Therefore, you can have the shortest $rs$-path from $(r, 0) \to (s, 1)$, on the auxiliary graph, that is actually the shortest $rs$-walk on $D$ with length $l \equiv 1 \pmod 3$, by running a Breadth-First-Search on the auxiliary digraph with the mentioned arguments.

Finally, to get the walk in $D$, the function Relabeling($p_{\text{aux}}$) receives a path on the auxiliary digraph and relabel all the vertices and arcs to match with the original graph. If a vertex has form $(v, i), i \in \{0, 1, 2\}$, it will be relabeled to $v$. An edge from $((v, i), (u, j)), i \in \{0, 1, 2\}, j \in \{0, 1, 2\}$, will be relabeled to $(v, u)$. This will make sure the $rs$-path is the according $rs$-walk in the original graph.

**Solution 4.** First, it is impossible to have both objects at the same time. If you have a matching that saturates $A$, you can build a set of pairs $(a, b), a \in A$, $b \in B$ and $a$ is a neighbor of $b$, such that every element of a pair is distinct from every other element of every other pair. Therefore, if there is a matching that saturates $A$, the inequality $|S| \leq |N(S)|$ holds for every $S \subseteq A$. The following algorithm solves the problem.

**Algorithm 3** Matching-or-Hall-Set $(G, (A, B))$

```
1: M, K ← Konig's-Algorithm (G, (A, B))
2: if |M| = |A| then
3:     return M
4: end if
5: for each v ∈ A do
6:     if v ∉ M then          ▷ this can be done in constant time by using an auxiliary array
7:         S ← ∅              ▷ S will be the subset described in the question
8:         π, d ← Breadth-First-Search(G, v)
9:         for each u ∈ V(G) do
10:            if (d[u] < +∞) and (u ∈ A) then
11:                S ← S ∪ u
12:            end if
13:        end for
14:        return S
15:    end if
16: end for
```

By corollary 20, the call of Konig's algorithm with a bipartite graph returns $(M, K)$, where $M$ is a maximum matching of $G$ and $K$ is a minimum vertex cover of $G$. It is correct because if the connected component always go back to the set A if it goes to the set B, because v and the subsequent are only connected to vertices on the matching, therefore this vertex is connected to a vertex in A.

The algorithm above runs Konig's algorithm, check if $M$ saturates $A$ and returns M if the verification is true. If it is not, the algorithm finds a vertex that it is not an end of any edge on $M$, and return the component (connected). To prove its correctness, I will take into account that, if $M$ does not saturate $A$, given that $M$ is a maximum matching of $G$, then no other matching saturates $M$. By that, if the algorithm does not return the matching in line 3, I should be able to return a subset $S \subseteq A$ such that $|S| > |N(S)|$.

If $M$ does not saturate $A$, there is at least on vertex $v$ that there is no edge in $M$ with an end in $v$. On line 6, when we find such $v$ we will run a Bradth-First-Search and after that we build a set with all the vertices that we can reach from $v$ and that lies on $A$. This set is a subset $S \subseteq A$ such that $|S| > |N(S)|$. To prove why that is true we need to go through some cases:

- If the vertex $v$ does not have a neighbor the only element in $S$ will be $v$ and then $S$ satisfy the conditions.
- If $v$ has a neighbor $u$ (which lies in $B$ because it is bipartite graph), $u$ will have at least one neighbor $w$ (which lies in $A$ because it is bipartite graph) because otherwise the matching could have another edge, but this is a contradiction because it is a maximum matching of $G$. It is clear that $w, u \in M$.

If $v$ has a neighbor $u$ (which lies in $B$ because it is bipartite graph), $u$ will have at least one neighbor $w$ (which lies in $A$ because it is bipartite graph) because otherwise the matching could have another edge, but this is a contradiction because it is a maximum matching of $G$. It is clear that $w, u \in M$. If $w$ does not have a neighbor This it will go back to the set A because if it was not the case the matching would not be the maximum matching.

On line 1 we run Konig's algorithm that was analyzed in class. Lines 2, 3 and 4 can be done in linear time using an array to mark each vertex in $A$ that the arcs on the matching has an end and then go through this array to verify $M$ saturates $A$. The FOR loop on line 5 runs at

most $|A|$ times, but the IF on line 6 is true only one time, because if it is true, latter will have a RETURN at line 14. Given that, we will run one BFS ($O(|V(G)| + |E(G)|)$) and the FOR at line 9 will run one time ($O(|V(G)|)$). Thus, the time of Konig's algorithm defines the time of the algorithm 3, ($O(|V(G)|(|V(G)| + |E(G)|))$).